

Ejercicio 6 — Agrupar dueños por tipo de mascota

```
27 tipo_mascota(Mascota, perro) :-
28     perro(Mascota).
29
30 tipo_mascota(Mascota, gato) :-
31     gato(Mascota).
32
33 tipo_mascota(Mascota, ave) :-
34     ave(Mascota).
35
36
37 % Obtener los dueños de un tipo específico de mascota
38
39 dueños_por_tipo(Tipo, Personas) :-
40     bagof(Persona,
41           Mascota*(dueño(Persona, Mascota),
42                   tipo_mascota(Mascota, Tipo)),
43           Personas).
44
45
46 % Obtener todos los grupos
47
48 grupos_dueños(Grupos) :-
49     bagof(Tipo=Personas,
50           dueños_por_tipo(Tipo, Personas),
51           Grupos).
```

Output:

```
dueños_por_tipo(perro, Personas).
Personas = [ana, luis, pedro]
dueños_por_tipo(gato, Personas).
Personas = [ana, luis, julia]
dueños_por_tipo(ave, Personas).
Personas = [maria]
grupos_dueños(Grupos).
Grupos = [ave-[maria], gato-[ana, luis, julia], perro-[ana, luis, pedro]]
grupos_dueños(Grupos).
```

Ejercicio 7 — Personas con al menos una mascota

Ejercicio 8 — Todas las mascotas tienen un dueño

```
1 gato(misu).
2 gato(luna).
3 gato(chanel).
4 gato(orion).
5
6 ave(piolin).
7
8 dueño(ana, firulais).
9 dueño(ana, misu).
10
11 dueño(luis, luna).
12 dueño(luis, orion).
13 dueño(luis, firulais).
14
15 dueño(maria, piolin).
16
17 dueño(julia, chanel).
18
19 dueño(pedro, bruno).
20
21 mascota(Mascota) :- perro(Mascota).
22
23 mascota(Mascota) :- gato(Mascota).
24
25 mascota(Mascota) :- ave(Mascota).
26
27 todas_las_mascotas_tienen_dueño :- forall(mascota(Mascota), dueño(_, Mascota)).
```

Output:

```
todas_tienen_mascota.
true
todas_las_mascotas_tienen_dueño.
false
mascota(X).
X = firulais
X = bruno
X = max
X = misu
X = luna
X = chanel
X = orion
X = piolin
mascota(X).
```

Ejercicio 9 — Amantes de los animales



```

5 gato(misu).
6 gato(luna).
7 gato(chanel).
8 gato(orion).
9
10 ave(piolin).
11
12 dueño(ana, firulais).
13 dueño(ana, misu).
14
15 dueño(luis, luna).
16 dueño(luis, orion).
17 dueño(luis, firulais).
18
19 dueño(maria, piolin).
20
21 dueño(julia, chanel).
22
23 dueño(pedro, bruno).
24 amante_animales(Persona) :-
25     dueño(Persona, Perro),
26     perro(Perro),
27     dueño(Persona, Gato),
28     gato(Gato).
29
30 amantes_animales(Personas) :-
31     findall(Persona, amante_animales(Persona), Personas).

```

amantes_animales(Personas).

Personas = [ana, luis, luis]

?- amantes_animales(Personas).

Examples History Solutions

Ejercicio 10 — Mascotas compartidas



```

5 gato(misu).
6 gato(luna).
7 gato(chanel).
8 gato(orion).
9
10 ave(piolin).
11
12 dueño(ana, firulais).
13 dueño(ana, misu).
14
15 dueño(luis, luna).
16 dueño(luis, orion).
17 dueño(luis, firulais).
18
19 dueño(maria, piolin).
20
21 dueño(julia, chanel).
22
23 dueño(pedro, bruno).
24 mascota_compartida(Persona1, Persona2, Mascota) :-
25     dueño(Persona1, Mascota),
26     dueño(Persona2, Mascota),
27     Persona1 @< Persona2.
28 mascotas_compartidas(Mascotas) :-
29     setof(Persona1, Persona2, Mascota),
30     mascota_compartida(Persona1, Persona2, Mascota),
31     Mascotas.

```

mascotas_compartidas(Mascotas).

Mascotas = [ana,luis,bruno]

?- mascotas_compartidas(Mascotas).

Examples History Solutions



```

1 gato(mina).
2 gato(luna).
3 gato(chanel).
4 gato(orion).
5
6 ave(piolin).
7
8 dueño(ana, luna).
9 dueño(ana, firulais).
10 dueño(ana, misu).
11
12 dueño(luis, luna).
13 dueño(luis, orion).
14 dueño(luis, firulais).
15
16 dueño(maria, piolin).
17
18 dueño(julia, chanel).
19
20 dueño(pedro, bruno).
21
22 mascota_compartida(Persona1, Persona2, Mascota) :-
23     dueño(Persona1, Mascota),
24     dueño(Persona2, Mascota),
25     Persona1 @< Persona2.
26
27 mascotas_compartidas(Mascotas) :-
28     setof((Persona1, Persona2, Mascota),
29         mascota_compartida(Persona1, Persona2, Mascota),
30         Mascotas).
31

```

```

mascotas_compartidas(Mascotas).
Mascotas = [[ana,luis,firulais]]
mascotas_compartidas(Mascotas).
Mascotas = [[ana,luis,firulais], [ana,luis,luna]]
? mascotas_compartidas(Mascotas).

```

Examples History Solutions

Parte 8 - Problema de la rana



```

1 % Coordenadas de las ubicaciones: ubicacion(ID, X, Y).
2 ubicacion(orilla_inicial, 0, 5).
3 ubicacion(piedra1, 2, 4).
4 ubicacion(piedra2, 5, 6).
5 ubicacion(piedra3, 8, 4).
6 ubicacion(piedra4, 5, 0).
7 ubicacion(orilla_final, 10, 5).
8
9
10
11 % Capacidad de la rana: distancia máxima de salto.
12 salto_maximo(4.0).
13
14 siguiente_estado(LugarActual, LugarDestino) :-
15     ubicacion(LugarActual, X1, Y1),
16     ubicacion(LugarDestino, X2, Y2),
17     LugarActual \= LugarDestino,
18     salto_maximo(Max),
19     Distancia is sqrt((X2-X1)*(X2-X1) + (Y2-Y1)*(Y2-Y1)),
20     Distancia <= Max.
21
22 dfs(LugarActual, LugarObjetivo, Visitados, Camino) :-
23     LugarActual = LugarObjetivo,
24     reverse(Visitados, Camino).
25
26 dfs(LugarActual, LugarObjetivo, Visitados, Camino) :- siguiente_estado(LugarActual, LugarDestino),
27     rana_puede_llegar(Camino) :- dfs(orilla_inicial, orilla_final, [orilla_inicial],

```

```

rana_puede_llegar(Camino).
Camino = [orilla_inicial, piedra1, piedra2, piedra3, orilla_final]
false
? rana_puede_llegar(Camino).

```

Examples History Solutions

Parte 9 - Problema de Batman vs Villanos

Program

Program

+

46

):-

47

member(Poder, Debilidades),

48

power_list(Poderes),

49

member(power(Poder, Dano, Costo), Poderes),

50

Energia >= Costo,

51

NuevaEnergia is Energia - Costo,

52

NuevaVida is Vida - Dano,

53

{

54

NuevaVida =< 8 ->

55

NuevosVillanos = RestoVillanos

56

;

57

NuevosVillanos = [

58

villain(Nombre, NuevaVida, Debilidades)

59

| RestoVillanos

60

]

61

};

62

63

64

% PREDICADO PRINCIPAL

65

66

batman_can_win(EnergiaMaxima) :-

67

power_list(Superpoderes),

68

villain_list(Villanos),

69

EstadoInicial = estado(Villanos, Superpoderes, EnergiaMaxima),

70

dfs(EstadoInicial, [EstadoInicial]).



batman_can_win(100).

false

batman_can_win(100).

Examples History Solutions